VSCode PR Review Plugin

Value Chain & KPI Framework

A strategic narrative for every layer of the organisation

Higher Mgmt | Mid Mgmt | Developers | PR Reviewers | New Joinees

Platform Engineering | Enterprise DevOps

What Is the Plugin?

A VSCode extension that brings AI-assisted PR review directly into the developer's IDE — and silently generates the organisation's Copilot ROI evidence as a side effect.

Self Review

Developer reviews their active file in IDE before raising a PR. Catches own issues — no context switch.

PR Review

AI reviews the full diff between source and destination branches. Comments posted directly to Bitbucket PR.

Jira AC + PR Review

Acceptance criteria from Jira combined with PR diff — AI validates code against the story definition.

7 Parameters stored in IDE

- Bitbucket DC URL
- Jira URL
- Project
- Repository
- Source branch
- Dest branch
- Basic auth (user/pwd)

Output → Review comments posted to Bitbucket PR

The Strategic Insight

This plugin doesn't have ***one*** value story.

It has ***five*** — one for each layer of the organisation.

And every layer's value **feeds the layer above it.**

The data is generated silently — as a side effect of developers doing their normal work.
No extra process. No overhead. No surveys. Just evidence.

"6 months ago, leadership asked teams to pilot Copilot.
This plugin is the only thing in the org that can tell you whether that pilot worked."

Five Value Chains — One Connected System

Each layer gets a different value. Each layer's actions create evidence for the layer above.

Higher Mgmt	Copilot licence spend justified with live adoption data	**ROI dashboard — not a survey**
Mid Mgmt	PR checklist + pattern data shows recurring issues per team	**Targeted coaching, fewer defects**
Developer	Self-review in IDE before raising PR — fewer review bounces	**Faster merge, less rework**
PR Reviewer	AI pre-screens diff — human focuses on logic not style	**Review fatigue drops, senior time freed**
New Joinee	Jira AC + real code teaches org standards without hand-holding	**Faster ramp, senior time saved**

All five chains converge → Mean time to merge ↓ | Defect rate ↓ | Copilot adoption % | Onboarding ramp time ↓ | Licence ROI ✓

KPIs — Higher Management (CTO / VP Engineering)

Their question: "We paid for Copilot. Is anyone actually using it? Did it help?"

KPI 1 | Copilot Plugin Adoption Rate

📡 Plugin telemetry — unique active users per week ÷ total licensed seats

Target: >60% of licensed devs active within 30 days

KPI 2 | Reviews Triggered per Project / Repo

📡 Bitbucket REST API — count of plugin-labelled PR comments

Target: Week-on-week upward trend across teams

KPI 3 | Review Type Distribution

📡 Plugin event log — self / PR / Jira+PR split per user

Target: Jira+PR usage grows as team maturity increases

KPI 4 | Copilot ROI Index

📡 Composite: adoption rate × quality delta × time-to-merge saved

Target: Positive index by month 3 — CTO-visible dashboard

KPIs — Mid Management (Delivery Leads / Engineering Managers)

Their question: "Which teams have review gaps? Where do defects repeatedly come from?"

KPI 1 | PR Checklist (TASK) Completion Rate

📡 Bitbucket TASK API — checked vs unchecked before merge

Target: >90% tasks completed before merge (merge gate enforces this)

KPI 2 | Recurring Issue Category Rate

📡 BB PR comments tagged by plugin label — category mining over time

Target: Top 3 issue types per team identified by month 2

KPI 3 | MRA (Mandatory Review Action) Resolution Rate

📡 BB TASK completion correlated with merge event

Target: 100% — merge gate enforces; non-negotiable

KPI 4 | Review Cycle Count per PR

📡 Bitbucket PR activity — comment rounds before approval

Target: Reduction of 1+ review cycle vs pre-plugin baseline

KPIs — Developers (Code Authors)

Their question: "Is this actually saving me time or just adding steps?"

KPI 1 | Self-Review Usage Before PR Raise

📡 Plugin event log — self-review session followed by PR created within 2 hrs

Target: >50% of PRs preceded by self-review within 8 weeks

KPI 2 | PR Rework Commits

📡 Bitbucket — commits pushed after PR raised (rework signal)

Target: 20% reduction vs pre-plugin baseline

KPI 3 | Time PR Raise → First Approval

📡 Bitbucket created_at vs approved_at across plugin-touched PRs

Target: 15–25% faster time-to-approval

KPI 4 | In-Plugin Satisfaction Signal

📡 Thumbs up/down after each review session (lightweight, in-IDE)

Target: >70% positive signal by month 2

KPIs — PR Reviewers (Senior Developers)

Their question: "Am I spending less time on obvious issues? Are incoming PRs actually cleaner?"

KPI 1 | Human Reviewer Comment Volume per PR

Bitbucket — human vs AI plugin-label comment split

Target: Human comment count falls as AI pre-screens more

KPI 2 | Time Reviewer Spends per PR

Bitbucket activity — first view timestamp to approval timestamp

Target: 20% reduction in time-to-approve

KPI 3 | Trivial / Style Issues by Human Reviewers

BB comment categorisation — style/lint comments by author type

Target: Near-zero trivial comments from human reviewers

KPI 4 | Checklist Task Dispute Rate

Bitbucket task reopen events — reviewer unchecks a completed task

Target: <5% dispute rate — confirms checklist is well-calibrated

KPIs — New Joinees (Onboarding Developers)

Their question: "Am I raising better PRs than week 1? Am I learning without blocking seniors?"

KPI 1 | First PR Approval Time

Bitbucket — new joinee's first PR created_at vs approved_at

Target: Measurably faster than pre-plugin new joinee cohort

KPI 2 | New Joinee PR Rejection Rate

Bitbucket — PRs needing >2 review cycles in first 60 days

Target: 30% lower than historical pre-plugin baseline

KPI 3 | Senior Dev Time on New Joinee PRs

Bitbucket reviewer activity — senior comment count on NJ PRs

Target: Downward trend from week 4 onward

KPI 4 | Jira AC Review Mode Adoption

Plugin event log — Jira+PR usage frequency by new joinee accounts

Target: >80% of new joinee PRs use Jira+PR mode

Business-Level KPIs — What All Chains Roll Up To

Provable with Bitbucket API data + plugin event log alone. No new tooling needed.

Mean Time to Merge (MTTM)

📡 BB: created_at → merged_at across all plugin-touched PRs

✓ 20–30% reduction in 90 days

Defect Escape Rate

📡 BB issues raised post-merge vs pre-plugin baseline

✓ Measurable reduction by quarter 2

Copilot Active Usage Rate

📡 Plugin telemetry: unique users / week ÷ total licensed

✓ >70% of licensed seats active monthly

Cost per Review Cycle Saved

📡 (Avg senior dev hourly cost) × (review cycles reduced)

✓ Plugin pays for itself in <6 months

Onboarding Ramp Time

📡 BB: new joinee first PR approval delta vs pre-plugin cohort

✓ Quantified reduction — HR-reportable metric

Where Does the Data Come From?

Every KPI across all five value chains is derivable from two sources already available in the organisation.

Bitbucket REST API

- PR created_at, approved_at, merged_at timestamps
- PR comment count (human vs plugin label split)
- TASK API — checklist completion before merge
- Commit count after PR raise (rework signal)
- Reviewer activity — first view to approval
- PR labels — identify plugin-touched reviews
- Postman-provable: full review chain per PR ID

Plugin Telemetry Event Log

- User ID + session timestamp per review
- Review type used (self / PR / Jira+PR)
- Project + repo per session
- Self-review → PR raise correlation
- New joinee vs veteran dev segmentation
- Satisfaction signal (thumbs up/down)
- Adoption rate: active users ÷ licensed seats

Zero additional tooling or process required — data is generated silently as developers work normally

The Bottom Line

01	Five value chains — one for every layer of the org, from CTO to new joinee.
02	Every chain has measurable KPIs — sourced from Bitbucket API and plugin telemetry.
03	Labels + TASK checklist = hard evidence, not surveys. Merge gate enforces review quality.
04	Plugin is the org's Copilot ROI engine — the only tool that proves whether the 6-month pilot worked.
05	Zero overhead. Data generated silently as developers do normal work.